FANCODE BACKEND ENGINEERING ASSESSMENT

Live Sports Match Data Aggregator

Complete Working Solution: Setup, Run & Verification Guide

Primary Setup: Docker Compose (Containerized, Zero Local Dependencies)
Secondary Setup: Native Maven Wrapper (Java 21 LTS)
Interactive Tool: SpringDoc OpenAPI 3.0 / Swagger UI (Browser Testing)
Automated Tests: 11/11 Tests Passing (Unit, Integration & Crash Recovery)
Durable Persistence: File-based H2 Database surviving JVM restarts

Author: Deepak Agrawal • Date: November 2026
Repository Path: `C:\Users\Deepak\OneDrive\Desktop\fancode-aggregator`

1. Setup & Execution Guide

The service is completely self-contained. A reviewer can launch and test the entire aggregator, both mock providers, the persistent database, and the interactive Swagger UI using either Docker or the Maven wrapper.

Option A: Docker-Based Setup (Preferred & Recommended)

Uses a multi-stage Docker build with Eclipse Temurin Java 21 Alpine JRE. Requires zero Java or Maven installation on the host machine.

```
# 1. Navigate to the project directory:
cd C:\Users\Deepak\OneDrive\Desktop\fancode-aggregator

# 2. Build image and start the container:
docker compose up --build

# (Optional: Run in background mode)
docker compose up -d --build
```

Container Specifications:

- Exposes HTTP port `8080` on `http://localhost:8080`.
- Mounts host directory `./data` to `/app/data` for durable, crash-proof H2 persistence.
- Includes container health checks querying `wget -qO- http://localhost:8080/matches`.

```
# To stop and remove the container:
docker compose down
```

Option B: Native Local Setup via Maven Wrapper

If running natively on a host machine with Java 21 installed, use the included Maven wrapper without installing Maven:

```
# On Windows (PowerShell / Command Prompt):
.\mvnw.cmd clean spring-boot:run

# On Linux / macOS:
./mvnw clean spring-boot:run
```

Active Service Endpoints

Resource	URL	Description
Interactive Swagger UI	`http://localhost:8080/swagger-ui/index.html`	Full browser UI to execute every API, trigger scenarios, and view live results.
OpenAPI 3.0 JSON Spec	`http://localhost:8080/v3/api-docs`	Machine-readable OpenAPI definition.
Client Matches API	`http://localhost:8080/matches`	Summary list of matches with status and formatted scores.
H2 Database Console	`http://localhost:8080/h2-console`	Database browser (JDBC URL: `jdbc:h2:file:./data/fancodedb`, User: `sa`).

2. Verification & Walkthrough Instructions

Follow this verification walkthrough using either the **Swagger UI** in your browser or **curl** in your terminal.

Method 1: Interactive Verification via Swagger UI

1. Open your web browser and navigate to `http://localhost:8080/swagger-ui/index.html`.
2. Under **Mock Provider A (Push Simulation)**, expand `POST /mock/provider-a/play` → click **Try it out** → click **Execute**.
   Result: Instantly executes all 6 steps from the FanCode prompt worked scenario. Step 3 confirms `duplicate_ignored`.
3. Under **Client Read APIs**, expand `GET /matches/match-1001/score` → click **Try it out** → click **Execute**.
   Result: Returns live score `10/1` in `1.1` overs.
4. Under **Client Read APIs**, expand `GET /matches/match-1001/commentary` → click **Execute**.
   Result: Returns 4 commentary balls sorted chronologically (1, 2, 3, 4). Confirms late ball `b3` is sorted in position 3 and `b2` text is updated to revision 2 (`"out, review stays out"`).
5. Under **Mock Provider B (Poll Simulation)**, expand `POST /mock/provider-b/scenario/error_503` → click **Execute**.
   Then execute `POST /mock/provider-b/poll`.
   Result: Poller handles the simulated HTTP 503 error gracefully; stored scores and commentary remain completely intact.

Method 2: Command-Line Verification via Curl

Step 1: Execute the 6-Step Worked Push Scenario

```
curl -X POST "http://localhost:8080/mock/provider-a/play?matchId=a-match-1001"
```

Response Output:


```
[
  {"status":"accepted","deliveryId":"d1","message":"Event processed successfully"},
  {"status":"accepted","deliveryId":"d2","message":"Event processed successfully"},
  {"status":"duplicate_ignored","deliveryId":"d2","message":"Delivery ID already processed, ignored as idempotent duplicate
  {"status":"accepted","deliveryId":"d4","message":"Event processed successfully"},
  {"status":"accepted","deliveryId":"d3","message":"Event processed successfully"},
  {"status":"accepted","deliveryId":"d5","message":"Event processed successfully"}
]
```


Step 2: Verify Scoreboard

```
curl -s http://localhost:8080/matches/match-1001/score
```


```
{
  "status": "live",
  "innings": 1,
  "runs": 10,
  "wickets": 1,
  "overs": "1.1",
  "scoreFormatted": "10/1",
  "provider": "provider-a"
}
```


Step 3: Verify Commentary (Strict Sequence Order & In-Place Correction)

```
curl -s http://localhost:8080/matches/match-1001/commentary
```


```
[
  { "ballId": "b1", "sequence": 1, "revision": 1, "text": "four" },
  { "ballId": "b2", "sequence": 2, "revision": 2, "text": "out, review stays out" },
  { "ballId": "b3", "sequence": 3, "revision": 1, "text": "two runs" },
```

```
  { "ballId": "b4", "sequence": 4, "revision": 1, "text": "six" }
]
```

3. Automated Test Suite (Scenarios & Edge Cases)

The test suite contains 11 automated JUnit 5 / SpringBootTest test cases validating every mandatory scenario and edge case:

```
# Command to execute full test suite:
.\mvnw.cmd test

# On Linux / macOS:
./mvnw test
```

Build & Test Result: Tests run: 11, Failures: 0, Errors: 0, Skipped: 0 • **BUILD SUCCESS**

#	Scenario / Edge Case	Test Method	What is Verified	Status
1	**Ordered Ingestion**	`testOrderedIngestion`	d1 arrives with 4/0 (0.4), then d2 arrives with 4/1 (0.5). Verifies monotonic score progression.	PASS
2	**Duplicate Push Delivery**	`testDuplicatePushDelivery`	Resending d1 with identical deliveryId returns `duplicate_ignored`. DB ball count stays exactly 1.	PASS
3	**Older Snapshot After Newer; Late Ball in Order**	`testOlderSnapshotWithLateBallReturnedInSequence`	d4 arrives with 10/1 (1.1). Later d3 arrives with older snapshot 8/1 (1.0). Score stays 10/1, while late ball b3 is inserted between b2 and b4.	PASS
4	**Commentary Correction**	`testCommentaryCorrection`	b2 revision 2 replaces text "out" with "out, review stays out". Resending revision 1 afterwards is ignored.	PASS
5	**Repeated Identical Polls**	`testRepeatedIdenticalPolls`	Polling Provider B multiple times in unchanged state creates zero duplicate balls in the database.	PASS
6	**Stale Poll Response**	`testStalePoll`	Stored score is 104/2 (seq 16). Provider B returns stale snapshot 80/1 (seq 10). Stored score remains 104/2.	PASS
7	**Provider 503 Fault Tolerance**	`testProviderErrorLeavesLastGoodState`	Provider B simulates HTTP 503. Poller catches error gracefully; last good stored state (98/2) remains live.	PASS
8	**Durable Database Restart**	`testRestartAgainstSameDatabase`	Spring Context 1 writes match data and closes completely. Spring Context 2 boots from disk file and recovers 100% of data.	PASS
9	**Full Worked Scenario**	`testFullWorkedPushScenario`	End-to-end execution of d1, d2, d2 duplicate, d4 gap, d3 delayed, d5 revision 2.	PASS
10	**Unknown Match Handling**	`testUnknownMatchReturns404`	Requesting non-existent match ID returns RFC-compliant HTTP 404 with structured error JSON.	PASS
11	**End-to-End Webhook & Read**	`testWebhookAndClientReadFlow`	Validates HTTP POST /api/webhooks/provider-a and subsequent GET requests via Spring MockMvc.	PASS

4. Assumptions & Key Design Decisions

Assumptions Documented

1. **Cricket Engine Scope:** The service does **not** calculate runs, wickets, or overs by interpreting commentary text. Score snapshots (`runs/wickets` , `overs`) are authoritative facts emitted by providers and stored as-is.
2. **Provider Numbering Independence:** Provider A and Provider B number sequences independently. Those numbers are **never comparable across providers**.
3. **Ball Identity Stability:** Each ball has a stable `ballId` (e.g. `inn1-12.4`). Corrections retain the same `ballId` with an incremented `revision` counter.
4. **Single-Process Durability:** A single local process backed by an embedded file-based relational database constitutes a complete solution. Stored data survives JVM restarts and system reboots.
5. **Provider Failure Tolerance:** Temporary provider outages (such as HTTP 503 from Provider B) leave the last good stored match available to clients without data loss or exceptions.

Key Architectural Design Decisions

1. Embedded File-Based Database (H2)

Configured at `jdbc:h2:file:./data/fancodedb;DB_CLOSE_DELAY=-1` . Zero installation overhead for reviewers while guaranteeing ACID transactions and persistence across restarts.

2. Two-Tier Idempotency Model

Push: Fast-path uniqueness on `(provider, delivery_id)` returning `duplicate_ignored` .
Poll: Entity-level checks on `(sequence, revision)` .

3. Lexicographical Monotonicity

Snapshot candidate `(s2, r2)` is strictly newer than stored `(s1, r1)` iff:
`(s2 > s1) || (s2 == s1 && r2 > r1)` .
Older or identical snapshots are safely rejected.

4. Sequence-Gap Tolerant Commentary

Scores update immediately; commentary does not block scores. Client commentary is sorted by `sequence ASC` . Late balls automatically slot into position.

5. Deterministic Precedence

Configured primary provider (`provider-a`) is served first. If primary data is absent, secondary (`provider-b`) is served. Sequences are never compared across providers.

6. Zero-Setup Docker & Swagger

SpringDoc OpenAPI 3.0 provides an interactive web UI at `/swagger-ui/index.html` . Multi-stage Docker build ensures execution on any OS with one command.

Webhook Response Contract & Crash Recovery Protocol

HTTP Status	Response Body	Meaning	Provider Action
`200 OK`	`{"status": "accepted", "deliveryId": "..."}`	Event committed to persistent database.	None (Acknowledged).
`200 OK`	`{"status": "duplicate_ignored", "deliveryId": "..."}`	DeliveryId was previously processed. State unchanged.	None (Acknowledged).
`400 Bad Request`	`{"error": "...", "status": 400}`	Malformed payload (missing provider or deliveryId).	Fix payload; do not retry as-is.
`500 Error`	`{"error": "...", "status": 500}`	Database crash or unhandled error before commit.	**Retry delivery with the exact same `deliveryId`** .